```
print "Comparing '{$polishChars1}' to '{$polishChars2}'" . PHP_EOL;

var_dump(strcmp(
    mb_strtolower($polishChars1),
    mb_strtolower($polishChars2)
));

Output:
Comparing 'ŁŃņ' to 'łńŅ'
int(0)
```

The two strings contain the same characters, but in mixed case (as we can see in the first line of output). `mb_strtolower()` converts both to lowercase, and we use `strcmp()` for a straightforward binary comparison. The result is `0`, and we know our strings contain the same characters. Be warned that this will be many times slower than `strcasecmp()`, but it will yield correct results!

strncmp(), strncasecmp()

Similar to `strcmp()`, `strncmp()` will do a byte-level comparison of two strings starting with the first byte. The function expects exactly three arguments: two strings and an integer value indicating how many bytes to compare. The function returns `-1`, `0`, or `1` if the first string is, respectively, less than, equal to, or greater than the second. The input strings can be of arbitrary length but must have a minimum length matching the third argument. Otherwise, it works in the same way, returning integer `-1`, `0`, or `1`:

```
var_dump(strncmp("Fender Stratocaster", "Fender Telecaster", 5));
var_dump(strncmp("Fender Stratocaster", "Fender Telecaster", 8));
var_dump(strncmp("Fender\x00Stratocaster", "Fender\x00Telecaster", 8));

Output:
int(0)
int(-1)
int(-1)
```

The third example above demonstrates the binary safety of `strncmp()`, a return value of `-1` showing that the function was able to compare the strings beyond the null byte. Multi-byte strings can be used with `strncmp()`, but the comparison is byte-level: the output of the function can be trusted for testing equality but should not be used